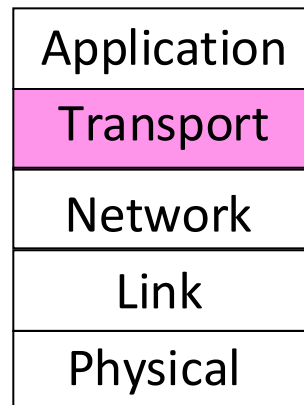


Capítulo 3

Capa de Transporte Generalidades e introducción a TCP



Capa de transporte

Propósito y funciones principales

- **¿Qué hace la capa de transporte (CT)?**
 - Provee **comunicación lógica entre procesos** de aplicación en diferentes hosts.
 - Es decir, los procesos se comunican como si estuvieran directamente conectados.
 - La capa de red (CR) solo comunica hosts; la CT va más allá: **comunica procesos**.
 - Se implementa **solo en los sistemas finales** (no en routers intermedios).
- **Mejora los servicios de la capa de red**
 - Ej.: **retransmite paquetes perdidos** en redes no orientadas a la conexión.
 - Ej.: **regula la tasa de transmisión** ante congestión en la red.

Capa de transporte

Ubicación y componentes

- **¿Dónde se ejecuta la CT?**
 - Se ejecuta **por completo en los hosts/sistemas finales** (computadoras de origen y destino).
 - Los **routers y switches intermedios no participan**: solo reenvían paquetes sin inspeccionar la capa de transporte.
- **Relación con la capa de red (CR)**
 - La CT **depende de la CR** (IP) para mover paquetes entre hosts, pero agrega: entrega confiable, control de flujo y multiplexación.
- **Entidad de transporte (ET)**
 - Es el **software (o hardware)** que implementa la lógica de la CT en cada host.
 - Ejemplo: en Linux, la ET de TCP es parte del kernel del sistema operativo.

Capa de transporte

Motivación para su estudio

- ¿Por qué conviene estudiar la capa de transporte?
 - **Diseño de aplicaciones:** al desarrollar una aplicación de red, es necesario elegir el protocolo de transporte adecuado según los **requisitos** (confiabilidad, velocidad, etc.).
 - **Aplicaciones eficientes:** conocer cómo funciona la CT permite escribir código que **aprovecha mejor la red**.
 - **API de sockets:** para usar sockets correctamente, hace falta entender los protocolos de transporte subyacentes (**TCP, UDP**).
 - **Innovación:** para diseñar nuevos protocolos o mejorar los existentes (ej.: **QUIC**).

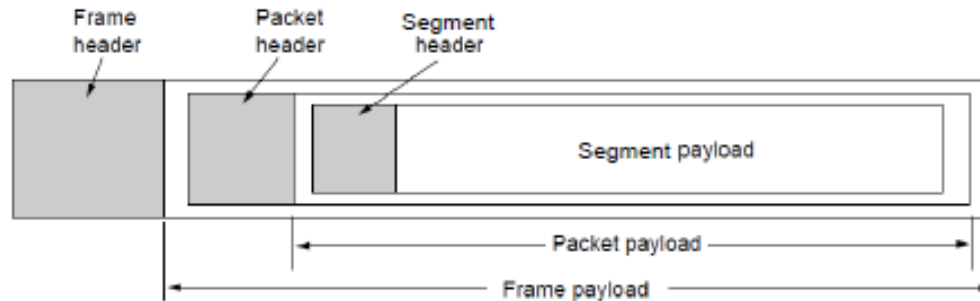
Capa de transporte

Problemas que resuelve

- La CT soluciona problemas clave de la comunicación en red:
 - **Confiabilidad:** uso de **temporizadores** y **retransmisiones** para recuperar paquetes perdidos.
 - **Direccionamiento:** **identificar el proceso** correcto en el host destino.
 - El proceso destino podría no estar activo, o el cliente podría no saber cuál usar.
 - **Eficiencia:** uso de **búferes** para una comunicación confiable sin desperdiciar recursos.
 - **Control de flujo:** evitar que un emisor rápido sature a un receptor lento.
 - **Control de congestión:** regular la cantidad de paquetes inyectados a la red: evitar que los búferes de los nodos intermedios se saturen y pierdan (hagan *drop* de) paquetes.
 - Si la CR pierde paquetes por congestión, la CT puede compensarlo.

Capa de transporte

Segmentos y encapsulamiento



- **Segmento** = unidad de datos del protocolo de transporte (TPDU).
- Los **segmentos** se encapsulan en **paquetes** (CR) y luego en **tramas** (capa de enlace).
 - Encapsulamiento = agregar un encabezado de la capa inferior al contenido de la capa superior, como una carta dentro de un sobre dentro de una caja.
- **Confirmaciones de recepción (ACK)**: el receptor notifica al emisor que los datos llegaron bien.
- **¿Qué se confirma?** Tanto paquetes de datos como paquetes de control (ej.: establecimiento de conexión).

Capa de transporte

Entrega confiable y ordenada

- **Problema: la red no garantiza orden ni entrega**
 - La capa de red (CR) puede **perder paquetes** (drop por búffer lleno o en la capa de enlace, perdidos por errores de canal), **duplicarlos** o entregarlos **fuera de orden**.
 - La CT debe (o puede) solucionar esto y es responsable de garantizar (o no):
 - **Entrega efectiva** de los segmentos al host destino.
 - **Entrega ordenada**: que los datos lleguen en el mismo orden en que fueron enviados por la capa de aplicación.
- **¿Cómo lo logra?**
 - Mediante mecanismos como números de secuencia, ACKs y temporizadores (ver siguiente slide).

Capa de transporte

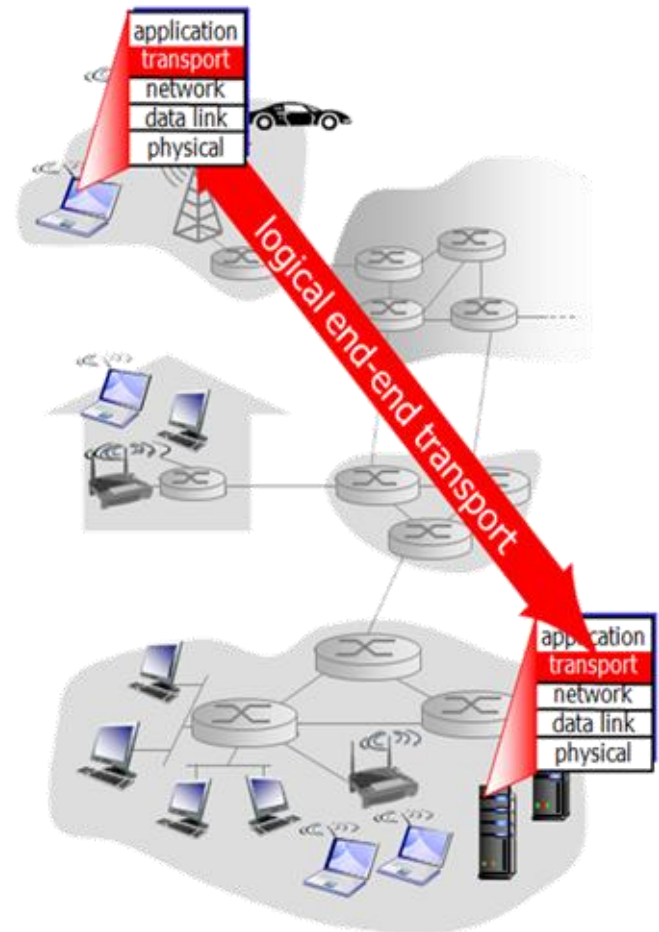
Mecanismos de entrega confiable

- **Solución: mecanismos para entrega ordenada y confiable**
 - 1. El emisor asigna **números de secuencia** a cada segmento, respetando el orden del flujo de datos.
 - 2. Al enviar un segmento, dispara un **temporizador de retransmisión**.
 - 3. El receptor envía **confirmaciones de recepción (ACK)** por cada segmento recibido correctamente.
 - 4. Si el temporizador expira sin recibir ACK → el emisor **retransmite** el segmento.
 - 5. El receptor **reensambla en orden** los segmentos recibidos y los entrega a la capa de aplicación.

TCP

Introducción y objetivos

- **¿Qué es TCP?**
 - El protocolo de transporte más usado en Internet. Garantiza que los datos lleguen **completos, en orden y sin errores**, incluso cuando la red debajo (IP) no lo garantiza.
- **Modelo de servicio**
 - **Flujo de bytes confiable (stream):** la aplicación ve una «tubería» continua de bytes entre origen y destino.
 - **Adaptativo:** ajusta velocidad y retransmisiones según las condiciones de la red en tiempo real.
- **Entidad TCP (ETCP)**
 - Software en cada host que implementa TCP (generalmente en el kernel del SO).



TCP

Problemas que resuelve

- **TCP aborda los siguientes problemas:**
 - **Retransmisión de paquetes:** mediante números de secuencia, ACKs y temporizadores.
 - **Duración de temporizadores:** algoritmos complejos para fijar tiempos de espera adaptativos.
 - **Manejo de conexiones:** establecer y terminar conexiones entre pares de procesos.
 - **Direccionamiento:** identificar procesos en hosts remotos mediante puertos.
 - **Control de congestión:** regular la inyección de paquetes para no saturar la red.
 - **Control de flujo:** evitar que un emisor rápido desborde a un receptor lento.

TCP "se hace cargo" de todas las responsabilidades de la capa de transporte (otros como UDP no lo hacen, pero ganan en eficiencia y rapidez de entrega).

TCP

Flujos de datos y segmentación

- **¿Cómo maneja TCP los datos de la aplicación?**
 - La aplicación envía un flujo continuo de bytes a TCP. TCP no conoce los límites de los «mensajes» de la aplicación: solo ve **un stream de bytes**.
 - TCP **fragmenta** ese flujo en **segmentos** de hasta 64 KB porque la red tiene límites de tamaño por paquete.
 - Cada segmento se envía dentro de un **datagrama IP** independiente.
- **Flujo completo (de punta a punta):**
 - App origen → **ETCP fragmenta** → IP encapsula → red transporta → **ETCP destino reensambla** → App destino.
 - La app destino recibe el mismo flujo de bytes, como si estuviera conectada directamente.

TCP

Sockets y conexiones

- **Sockets**
 - Para usar TCP, tanto el cliente como el servidor crean **sockets** (puntos de acceso al servicio).
 - **Dirección de socket** = dirección IP + número de puerto.
- **Conexiones**
 - Se debe **establecer una conexión** explícita entre un socket emisor y uno receptor.
 - Un mismo socket puede tener **múltiples conexiones** simultáneas.
 - Cada conexión se identifica por el par: **(socket_origen, socket_destino)**.

TCP

Números de secuencia y segmentos

- **Números de secuencia en TCP**
 - Cada **byte** del flujo de datos tiene su propio **número de secuencia** de 32 bits.
 - Esto impone un límite teórico al tamaño del flujo: $2^{32} = \sim 4$ GB.
- **¿Para qué sirven los números de secuencia?**
 - Permiten al receptor enviar **confirmaciones de recepción (ACK)** precisas.
 - Permiten **detectar datos faltantes, duplicados y desordenados**.
- **Estructura de un segmento**
 - **Segmento** = encabezado TCP + datos (0 o más bytes).

TCP

Tamaño máximo de segmento (MTU)

- **¿Qué limita el tamaño de un segmento TCP?**
 - **Límite IP:** cada segmento debe caber en la carga útil del datagrama IP en la capa de red (máx. 65.515 bytes).
 - **MTU (Unidad Máxima de Transferencia):** cada red física impone un límite al tamaño de trama. El segmento debe caber en la MTU.
 - En la práctica, la MTU suele ser de **1500 bytes** (carga útil de Ethernet).
- **En resumen**
 - El tamaño máximo de segmento (MSS) es el menor entre el límite IP y la MTU de la red, descontando encabezados.
 - En Ethernet: MSS típico ≈ 1460 bytes (1500 - 20 IP - 20 TCP).

TCP

Temporizadores y retransmisiones

- **Problema: IP no garantiza la entrega**
 - La capa de red (IP) no garantiza que los datagramas se entregarán, ni que llegarán correctamente.
- **Solución de TCP: ACKs + temporizadores**
 - Si un segmento llega correctamente → el receptor envía un **ACK** (confirmación).
 - Si el **temporizador** expira sin recibir ACK → TCP retransmite el segmento.
 - TCP es responsable de gestionar los temporizadores y ejecutar las retransmisiones según sea necesario.

TCP

Reordenamiento de segmentos

- **Problema: los datagramas pueden llegar fuera de orden**
 - En redes de datagramas, cada paquete **puede tomar rutas diferentes** y llegar en **orden distinto al de envío**.
- **¿Por qué es un problema?**
 - La capa de aplicación, en muchos casos, necesita procesar los datos en el orden original de envío (p. ej., un video, un archivo binario).
- **Solución de TCP**
 - TCP usa los **números de secuencia** para **reensamblar** los segmentos en la secuencia correcta antes de entregarlos a la aplicación.

TCP

Mecanismo de ACK

- ¿Cómo funciona el mecanismo de ACK?
 - 1. Al enviar un segmento, el emisor inicia un **temporizador**.
 - 2. Al llegar al destino, la ETCP receptora responde con un segmento que contiene el **número de confirmación (ACK number)**.
 - El ACK number = *siguiente número de secuencia que espera recibir*.
 - El ACK puede ir acompañado de datos (si hay algo para enviar en el sentido inverso, en una comunicación bi-direccional).
 - 3. Si el temporizador expira antes de recibir el ACK → el emisor retransmite el segmento.

TCP

Desafíos del orden y retransmisión

- **Desafíos que TCP debe resolver eficientemente:**
- **1. Segmentos fuera de orden**
 - Ej.: los bytes 3072–4095 pueden llegar antes que los bytes 2048–3071.
 - **Consecuencia:** TCP debe almacenar los segmentos adelantados en un **búfer de reordenamiento** y esperar a que lleguen los faltantes antes de entregar a la aplicación.
 - Tampoco puede confirmar un hueco: si llegó 1–999 y 2000–2999, solo puede confirmar hasta 999.
- **2. Segmentos retardados**
 - Si un segmento tarda más que el temporizador, TCP lo retransmite (posiblemente de forma innecesaria si el original aún está en tránsito).
 - **Solución:** el receptor descarta duplicados usando los **números de secuencia** (ya tiene esos bytes → los ignora).

TCP

Retransmisiones y control de bytes

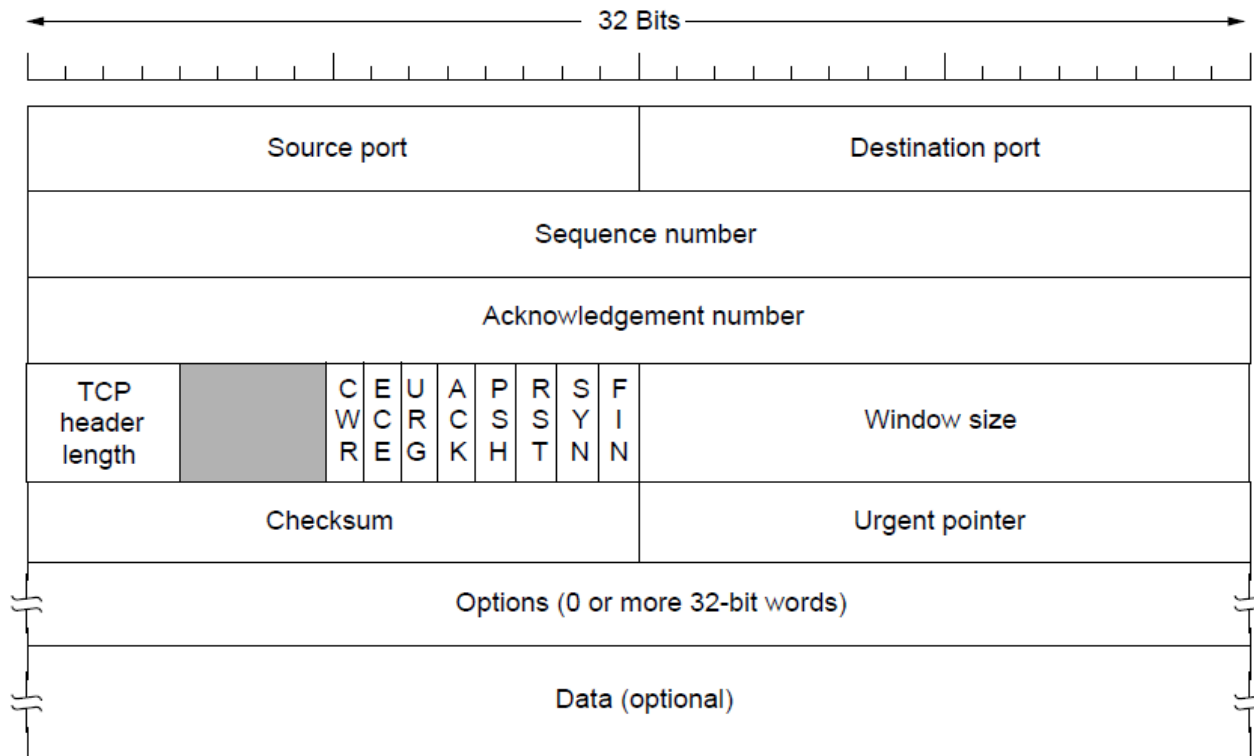
- **¿Qué pasa al retransmitir un segmento?**
 - Las retransmisiones pueden incluir **rangos de bytes diferentes** al segmento original.
- **¿Por qué?**
 - Al momento de retransmitir, la aplicación puede haber agregado nuevos datos al búfer de envío. TCP puede **combinar los bytes pendientes** en un nuevo segmento de tamaño diferente.
- **Consecuencia para el receptor**
 - El receptor no puede asumir que los segmentos llegan con los mismos rangos que fueron enviados originalmente. Debe llevar un **control byte a byte** de qué bytes se recibieron correctamente.

Segmentos TCP

Estructura del encabezado

Todo segmento TCP tiene tres partes:

1. **Encabezado fijo** — 20 bytes. Contiene puertos, números de secuencia/ACK, flags y control de flujo.
2. **Opciones** — Longitud variable (en palabras de 32 bits). Negocian parámetros como MSS.
3. **Datos** — Opcionales. Un segmento puede ser solo encabezado (ej.: un ACK puro).



Segmentos TCP

Puertos y direccionamiento

- **Segmentos sin datos**
 - Se usan para enviar **ACKs** y **mensajes de control** (SYN, FIN). Solo contienen el encabezado TCP, sin datos de aplicación.
- **Puerto de origen y puerto de destino**
 - Cada uno ocupa **16 bits** (valores de 0 a 65.535).
 - Puertos conocidos: 80 (HTTP), 443 (HTTPS), 22 (SSH), 25 (SMTP).
 - IP + puerto = **socket** (punto terminal único de 48 bits que identifica un proceso en un host).
 - El par **(socket_origen, socket_destino)** identifica de forma única cada conexión TCP.

Encabezado TCP

Campos de secuencia y ACK

- **Campo: Número de secuencia (32 bits)**
 - Identifica la posición del **primer byte** de datos del segmento dentro del flujo de bytes total.
 - Ej.: si el flujo empieza en byte 0 y un segmento contiene los bytes 1000-1499, su número de secuencia es 1000.
- **Campo: Número de confirmación (ACK number, 32 bits)**
 - Indica el **siguiente byte que el receptor espera recibir**.
 - Ej.: si recibió correctamente hasta el byte 1499, el ACK number será 1500.
 - Confirma implícitamente todos los bytes anteriores (ACK acumulativo).

Encabezado TCP

Flag ACK

- **Flag ACK (1 bit en el encabezado)**
 - Si **ACK = 1**: el campo «número de confirmación» es válido → el segmento confirma datos recibidos.
 - Si **ACK = 0**: el campo se ignora → no hay confirmación en este segmento.
- **Piggybacking: ACKs «gratis» con datos**
 - En la práctica, casi todos los segmentos (excepto el primer SYN) llevan **ACK = 1**, porque TCP aprovecha cada envío para confirmar lo recibido.
- **Ejemplo concreto**
 - A envía datos (seq=1000, 500 bytes) → B responde con datos propios **y ACK=1500** en el mismo segmento → confirma los 500 bytes de A sin un paquete extra.

Encabezado TCP

Longitud y opciones

- **Campo: Longitud del encabezado (4 bits)**
 - Indica el número de **palabras de 32 bits** en el encabezado TCP. Es necesario porque el encabezado tiene tamaño variable (por las opciones).
 - Mínimo: 5 palabras = 20 bytes (encabezado fijo, sin opciones).
 - Máximo: 15 palabras = 60 bytes (con opciones llenas).
- **Campo de opciones (longitud variable)**
 - Permite **negociar parámetros** entre emisor y receptor al inicio de la conexión.
 - **MSS**: tamaño máximo de segmento que acepta cada lado.
 - **Window Scale**: ampliar la ventana de recepción más allá de 64 KB.
 - **SACK**: confirmaciones selectivas (confirmar bloques no contiguos).
 - Máximo espacio para opciones: 40 bytes (60 - 20 del encabezado fijo).

Segmentos TCP

Flags del encabezado

1. **URG (Urgent)**: Indica que el segmento contiene datos urgentes que deben procesarse de inmediato. El campo **Urgent Pointer** acompaña este indicador y señala la posición en el flujo de datos donde terminan los datos urgentes.
2. **PSH (Push)**: Sirve para pedir al receptor que procese y entregue los datos inmediatamente al nivel superior (aplicación), en lugar de esperar a completar el buffer. Esto se usa en escenarios donde la inmediatez es clave.
3. **RST (Reset)**: Se utiliza para reiniciar una conexión. Se puede enviar, por ejemplo, cuando hay un error crítico en la comunicación o se quiere rechazar una conexión no deseada.
4. **Urgent Pointer**: Complementa el indicador **URG**. Su propósito es especificar la ubicación del último byte de datos urgentes dentro del segmento.
5. **CWR (Congestion Window Reduced)** y **ECE (Explicit Congestion Notification Echo)**: Relacionados con el manejo de congestión en la red. **CWR** indica que el transmisor ha reducido su ventana de congestión. **ECE** señala que el receptor ha detectado congestión a través de notificaciones explícitas.

Encabezado TCP

Ejercicio de revisión

- **Ejercicio de revisión:**

- 1. ¿Hasta cuántas palabras de 32 bits se pueden tener en un encabezado TCP? ¿Por qué?
- 2. ¿Hasta cuántas palabras de 32 bits puede ocupar el campo de opciones? ¿Cuántos bytes son?
- 3. Si un host recibió correctamente hasta el byte 4999 de un flujo TCP, ¿qué valor tendrá el campo ACK number en su próximo segmento? ¿Y qué valor tendrá el flag ACK?

- **Pistas:**

- Para las preguntas 1 y 2: revisá el campo «longitud del encabezado» (4 bits).
- Para la pregunta 3: recordá que el ACK number indica el siguiente byte esperado.

Ejercicios de repaso

Capa de transporte — Conceptos generales (1/4)

Ejercicio 1 — Verdadero o Falso

«La capa de transporte se ejecuta tanto en los hosts como en los routers intermedios.»

Ejercicio 2 — Verdadero o Falso

«TCP ve los datos de la aplicación como mensajes individuales y los transmite respetando sus límites.»

Ejercicio 3 — Multiple Choice

¿Qué es un «segmento» en el contexto de TCP?

- a) Un paquete de la capa de red
- b) La unidad de datos del protocolo de transporte (TPDU)
- c) Una trama de la capa de enlace
- d) Un datagrama IP completo

Ejercicios de repaso

Capa de transporte — Conceptos generales (1/4)

Ejercicio 1 — Verdadero o Falso

«La capa de transporte se ejecuta tanto en los hosts como en los routers intermedios.»

Respuesta: FALSO. Se ejecuta solo en los hosts/sistemas finales. Los routers solo reenvían paquetes sin inspeccionar la capa de transporte.

Ejercicio 2 — Verdadero o Falso

«TCP ve los datos de la aplicación como mensajes individuales y los transmite respetando sus límites.»

Respuesta: FALSO. TCP no conoce los límites de los mensajes de la aplicación: ve un flujo continuo de bytes (stream) y lo fragmenta en segmentos según convenga.

Ejercicio 3 — Multiple Choice

¿Qué es un «segmento» en el contexto de TCP?

- a) Un paquete de la capa de red
- b) La unidad de datos del protocolo de transporte (TPDU)
- c) Una trama de la capa de enlace
- d) Un datagrama IP completo

Respuesta: b) El segmento es la unidad de datos de la capa de transporte. Se encapsula dentro de un paquete IP (capa de red).

Ejercicios de repaso

Encabezado TCP — Cálculos y conceptos (2/4)

Ejercicio 4 — Completar (cálculo)

El campo «longitud del encabezado» tiene 4 bits. ¿Cuál es el número máximo de palabras de 32 bits que puede tener el encabezado TCP? ____

Ejercicio 5 — Completar (cálculo)

Si el encabezado fijo de TCP mide 20 bytes y el máximo es 60 bytes, ¿cuántos bytes pueden ocupar las opciones como máximo? ____

Ejercicio 6 — Verdadero o Falso

«Un socket se identifica únicamente con el número de puerto.»

Ejercicios de repaso

Encabezado TCP — Cálculos y conceptos (2/4)

Ejercicio 4 — Completar (cálculo)

El campo «longitud del encabezado» tiene 4 bits. ¿Cuál es el número máximo de palabras de 32 bits que puede tener el encabezado TCP? _____

Respuesta: 15. Con 4 bits el valor máximo es $2^4 - 1 = 15$. Eso equivale a $15 \times 4 = 60$ bytes.

Ejercicio 5 — Completar (cálculo)

Si el encabezado fijo de TCP mide 20 bytes y el máximo es 60 bytes, ¿cuántos bytes pueden ocupar las opciones como máximo? _____

Respuesta: 40 bytes. 60 bytes (máximo total) - 20 bytes (encabezado fijo) = 40 bytes para opciones.

Ejercicio 6 — Verdadero o Falso

«Un socket se identifica únicamente con el número de puerto.»

Respuesta: FALSO. Un socket = dirección IP + puerto. Dos hosts distintos pueden usar el mismo puerto (ej.: ambos en puerto 80), pero sus sockets son diferentes porque tienen distinta IP.

Ejercicios de repaso

Números de secuencia y ACK (3/4)

Ejercicio 7 — Completar (cálculo)

Un host recibió correctamente todos los bytes de un flujo TCP hasta el byte 4999. ¿Qué valor tendrá el campo ACK number en su próximo segmento? ____

Ejercicio 8 — Verdadero o Falso

«Si el flag ACK vale 0, el campo número de confirmación contiene un valor válido.»

Ejercicio 9 — Completar (cálculo)

Un enlace tiene MTU de 1500 bytes. IP ocupa 20 bytes y TCP ocupa 20 bytes. ¿Cuál es el MSS? ____

Ejercicios de repaso

Números de secuencia y ACK (3/4)

Ejercicio 7 — Completar (cálculo)

Un host recibió correctamente todos los bytes de un flujo TCP hasta el byte 4999. ¿Qué valor tendrá el campo ACK number en su próximo segmento? _____

Respuesta: 5000. El ACK number indica el siguiente byte esperado. Si recibió hasta el 4999, espera el 5000.

Ejercicio 8 — Verdadero o Falso

«Si el flag ACK vale 0, el campo número de confirmación contiene un valor válido.»

Respuesta: FALSO. Cuando ACK = 0, el campo se ignora. Solo es válido cuando ACK = 1.

Ejercicio 9 — Completar (cálculo)

Un enlace tiene MTU de 1500 bytes. IP ocupa 20 bytes y TCP ocupa 20 bytes. ¿Cuál es el MSS? _____

Respuesta: 1460 bytes. $MSS = MTU - IP - TCP = 1500 - 20 - 20 = 1460$ bytes.

Ejercicios de repaso

Flags, conexiones y opciones (4/4)

Ejercicio 10 — Multiple Choice

¿Qué flag del encabezado TCP se utiliza para iniciar una conexión?

- a) FIN b) RST c) SYN d) PSH

Ejercicio 11 — Multiple Choice

¿Para qué sirve la opción MSS en el encabezado TCP?

- a) Indicar el tamaño de la ventana de congestión
- b) Negociar el tamaño máximo de segmento que acepta cada lado
- c) Definir el número de secuencia inicial
- d) Establecer el tiempo de vida del paquete

Ejercicio 12 — Verdadero o Falso

«Una conexión TCP se identifica de forma única por un solo socket (IP + puerto).»

Ejercicios de repaso

Flags, conexiones y opciones (4/4)

Ejercicio 10 — Multiple Choice

¿Qué flag del encabezado TCP se utiliza para iniciar una conexión?

- a) FIN b) RST c) SYN d) PSH

Respuesta: c) SYN. SYN (synchronize) se usa para solicitar el establecimiento de una conexión TCP.

Ejercicio 11 — Multiple Choice

¿Para qué sirve la opción MSS en el encabezado TCP?

- a) Indicar el tamaño de la ventana de congestión
- b) Negociar el tamaño máximo de segmento que acepta cada lado
- c) Definir el número de secuencia inicial
- d) Establecer el tiempo de vida del paquete

Respuesta: b) MSS (Maximum Segment Size) se negocia al inicio de la conexión para que cada lado informe el tamaño máximo de datos que puede recibir por segmento.

Ejercicio 12 — Verdadero o Falso

«Una conexión TCP se identifica de forma única por un solo socket (IP + puerto).»

Respuesta: FALSO. Una conexión TCP se identifica por el par (socket_origen, socket_destino), es decir, dos sockets: uno en cada extremo.